



Politechnika Wrocławska

Maciej Kowal 2593853
Algorytmy Sortowania

Spis treści

1	Wprowadzenie	2
2	GitHub	2
3	Opis badanych algorytmów	2
3.1	Sortowanie przez Scalanie	2
3.2	Quicksort	2
3.3	Insert	3
3.4	Zarządzanie Pamięcią	3
4	Drzewo czerwono-czarne	3
4.1	Drzewa binarne	3
4.1.1	Przeszukiwanie drzew binarnych	3
4.2	BST czerwono-czarne	4
4.3	Teoretyczna złożoność obliczeniowa dla drzewa czerwono-czarnego .	5
5	Omówienie przebiegu eksperymentów	6
5.1	Tabela wyników pomiarowych	6
5.2	Drzewo czerwono-czarne	7
5.3	Merge sort	8
5.4	Quicksort	9
6	Bibliografia	9

1 Wprowadzenie

Celem laboratorium było stworzenie programu sortującego Filmy z bazy danych według ratingów. W celu dopasowania ratingów do tytułów konieczne było utworzenie samorównoważającego się drzewa przeszukiwań binarnych, dzięki któremu możliwe było znalezienie filmu według wartości $tconst$.

2 GitHub

link repozytorium:

<https://github.com/ckMako/algorZapasowy.git>

link dane:

<https://drive.google.com/file/d/1Jh5MxW2bE98XQY45bcMq7qrkVCKj-nil/view?usp=sharing>

3 Opis badanych algorytmów

3.1 Sortowanie przez Scalanie

Algorytm sortowania przez scalanie (merge sort) polega na podziale tablicy na dwie równe połowy, aż do uzyskania podtablic jednoelementowych, które są trywialnie posortowane. Następnie podtablice są scalane parami w posortowanej kolejności.

Złożoność czasowa wynosi $O(n \log n)$ w każdym przypadku (najlepszym, średnim i najgorszym), ponieważ tablica zawsze dzielona jest na równe połowy. Wymaga dodatkowej tablicy tymczasowej.

3.2 Quicksort

Algorytm quicksort polega na wyborze elementu zwanego pivotem, a następnie podziale tablicy na dwie części: elementy mniejsze od pivota i elementy większe od pivota. Operacja ta jest wykonywana dla obu części.

Złożoność czasowa wynosi $O(n \log n)$ w przypadku średnim i najlepszym, natomiast $O(n^2)$ w przypadku najgorszym, gdy pivot jest zawsze najmniejszym lub największym elementem. Zajmuje mniej pamięci niż merge sort, ponieważ nie wymaga dodatkowej tablicy tmp.

W programie zastosowano dodatkową tablicę, który wydziela dodatkowo elementy równe pivotowi, ponieważ w przypadku ratingów pojawia się dużo powtórzeń, co pogarsza złożoność obliczeniową. Rozwiązanie to znacząco przyspiesza sortowanie dla danych z wieloma duplikatami – w przypadku rankingów filmów,

gdzie wartości przyjmują jedynie 90 unikalnych wartości (od 1.0 do 10.0 z krokiem 0.1), czas sortowania 100 000 elementów skrócił się z 8716 ms do 24 ms.

3.3 Insert

Algorytm sortowania przez wstawianie (insert) polega na budowaniu posortowanej tablicy element po elemencie. Każdy kolejny element jest wstawiany na właściwą pozycję wśród już posortowanych elementów poprzez przesuwanie większych elementów w prawo, co jest wolne ponieważ w zmiennych tablicowych trzeba skopiować wszystkie dane.

Złożoność czasowa wynosi $O(n^2)$ w najgorszym przypadku, natomiast $O(n)$ w przypadku najlepszym, gdy dane są już posortowane.

3.4 Zarządzanie Pamięcią

W algorytmach stosujemy vector filmów oraz vector indeksów, sortujemy same indeksy, aby uniknąć kopiowania struktury danych jaką jest film. Ponadto, ponieważ vector jest tablicą dynamiczną, stosujemy metodę `.reserve()` w celu uniknięcia kopiowań podczas `.pushBack()`. Na początku planowałem stworzyć vector wskaźników, jednak dynamiczna alokacja pamięci jest bardzo wolna (`new`). Innym pomysłem było zapisanie obiektów w wektorze nr 1 i wskaźników w nr 2, jednak wówczas przy operacji `.pushBack()` może nastąpić kopiowanie danych (powiększenie pamięci dyn), w takim przypadku wszystkie wskaźniki stają się bezużyteczne.

4 Drzewo czerwono-czarne

W ramach laboratorium zaimplementowane zostało drzewo czerwono-czarne. Równoważy się dzięki utrzymywaniu stałej liczby węzłów czarnych w każdej gałęzi.

4.1 Drzewa binarne

Drzewa binarne (BST-binary search tree) to szczególny rodzaj drzewa jako struktury danych, każdy węzeł w drzewie wskazuje na 2 węzły niżej (dzieci), ponadto węzły przechowują wartość klucza, na podstawie której dokonuje się przeszukiwań binarnych., mogą także przechowywać blok danych dowolnego typu.

4.1.1 Przeszukiwanie drzew binarnych

Algorytm znajdowania określonej wartości w drzewie binarnym jest prosty i opiera się na operacji porównania. Na początku określamy wartość której szukamy (klucz), przeszukiwanie rozpoczynamy od korzenia, porównujemy dany klucz

z kluczem korzenia, jeżeli jest mniejsza to następnym elementem do porównania jest lewy syn węzła, jeżeli większa to następnie porównujemy prawego syna węzła, jeżeli jest równa, wówczas znaleźliśmy szukaną wartość. Wstawianie węzła o danym kluczu do drzewa odbywa się w ten sam sposób, tylko nie zatrzymujemy algorytmu po znalezieniu wartości równej kluczowi, lecz po dotarciu do węzła wskazującego na nullptr, wówczas dany węzeł zastępuje nullptr, jednak może to powodować wypaczanie drzewa, tj. drzewo zaczyna przypominać listę zamiast drzewa.

4.2 BST czerwono-czarne

Czerwono-czarne drzewa przeszukiwań są specjalnym typem drzew. Ich celem jest zapobieganie wypaczaniu gałęzi drzewa. W tym celu tworzy się nowe pole w węźle jakim jest kolor, kolor przyjmuje tylko wartości czarny i czerwony. Na podstawie kolorów węzłów określa się maksymalną długość każdej gałęzi. Główna zasada jest taka że ilość czarnych węzłów każdej gałęzi musi być taka sama. Aby jednak struktura działała musi przestrzegać również innych zasad:

Zasady których przestrzega drzewo Czerwono-czarne

1. Ilość węzłów czarnych w każdej gałęzi musi być taka sama, jeżeli któraś gałąź ja łamie, wówczas albo zmienia się kolory gałęzi wyżej, albo stosuje się rotacje lewo lub prawostronną (w lewostronnej prawy syn staje się ojcem, ojciec staje się lewym synem(mniejszy), natomiast lewy syn nowego ojca staje się prawym synem byłego ojca(wiekszy))
2. Korzeń jest zawsze czarny
3. Czarne węzły mogą mieć dzieci koloru czarnego lub czerwonego
4. Czerwone węzły nie mogą mieć dzieci koloru czerwonego(dzięki tej zasadzie różnica węzłów pomiędzy gałęziami będzie maksymalnie dwukrotna)
5. Nowe węzły są zawsze koloru czerwonego(w przeciwnym wypadku wszystkie węzły byłyby czarne co zmienia drzewo czerwono-czarne w zwykłe AVL i sprawia że kolorowanie jest bez sensu).

W celu zachowania powyższych właściwości drzewa czerwono-czarnego stosuje się różne algorytmy, tj. rotacji(lewo, prawo), kolorowania węzłów(jeżeli ojciec węzła czerwonego jest czerwony, wówczas ojcaojca koloruje się na czerwono, a jego synów na czarno, z gałęzi znika 1 węzeł czarny, ale jest zastępowany, robi się to do roota(root zawsze czarny)).

W celu uniknięcia błędów ochrony pamięci (nullprt-¿metoda()) stosuje się specjalny węzeł strażnika(NIL), wszystkie wskaźniki które miały by mieć wartość nullptr (lewi i prawi synowaie nowych węzłów, ojciec root) zastępuje się wskaźnikiem na

NIL, NIL jako pusty obiekt klasy węzeł(node) nie powoduje błędów ochrony pamięci(lewy i prawy syn oraz ojciec NIL to NIL)(NIL teoretycznie ma wszystkie pola węzła tylko puste).

4.3 Teoretyczna złożoność obliczeniowa dla drzewa czerwono-czarnego

Największą przewagą jaką mają BST nad listą np dwukierunkową jest zmniejszenie złożoności obliczeniowej podczas wyszukiwania, Jeżeli chcemy znaleźć element o danym kluczu, w liście wiązanej musimy porównać wartość klucza z każdym węzłem poprzedzającym co daje złożoność obliczeniową:

$$O(n)$$

natomiast w drzewie binarnym, po przejściu przez każdy węzeł odrzucamy połowę danych, dzięki czemu uzyskujemy:

$$O(\log_2 n)$$

W drzewie czerwono-czarnym złożoność jest trochę inna, ponieważ zależy od ilości węzłów czerwonych, niektóre gałęzie mogą być przez to 2 razy dłuższe od innych, zatem dla czerwono-czarnego BST mamy powyższą złożoność w najlepszym wypadku, natomiast w najgorszym:

$$O(2 \log_2 n)$$

Złożoności obliczeniowe dla różnych operacji w BST czerwono-czarnym

- **find** $O(\log n) - O(2 \log n)$ jak wyżej
- **insert** $O(\log n)$ musimy dotrzeć do wskaźnika na NIL
- **insert(liczba instancji)** trudno określić, ponieważ duży wpływ będzie miała ilość wywołań algorytmów rotate oraz fixColor(przemałuj) nad którym nie mamy kontroli.
- **remove** $O(\log n + C)$ taka sama jak w przypadku insert, musimy znaleźć element o danym kluczu, należy również naprawić drzewo, jednak zakładamy że to stały czas.
- **zmień wartość dla klucza** $O(\log n)$ to samo co w find, należy znaleźć wartość w drzewie

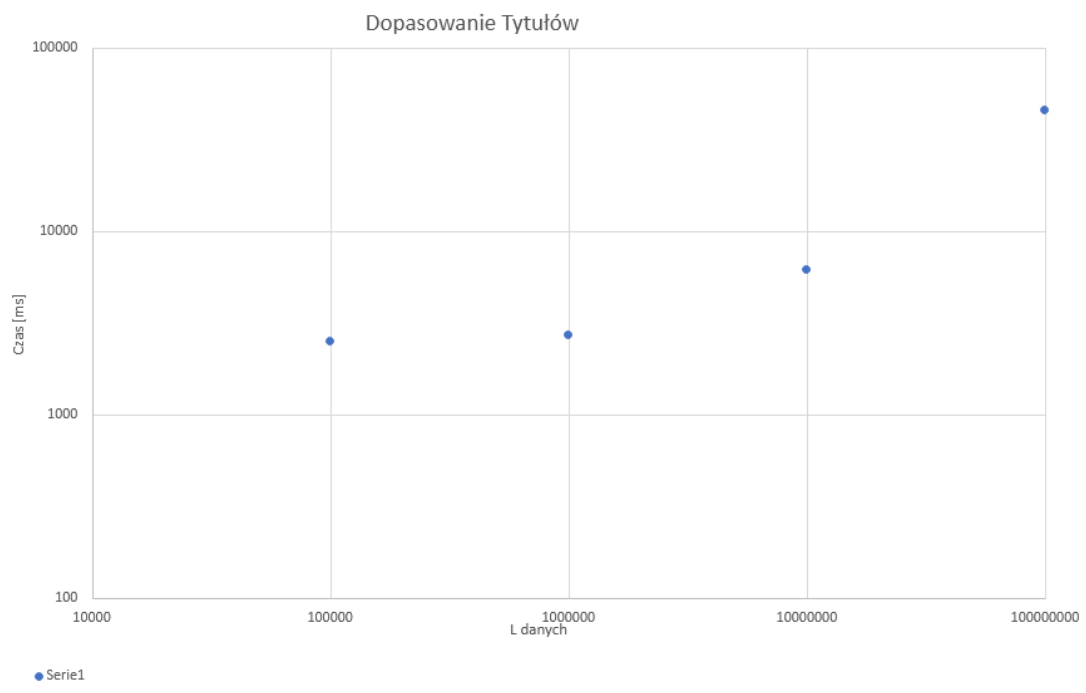
-
- **zmień wartość danego klucza** $O(\log n + C)$ algorytmy find + remove + insert.
 - **maksimum/minimum** $O(\log n)$ Najprostszy algorytm ze wyszukiwaniem, w celu znalezienia maksimum kierujemy się zawsze w prawo aż do węzła którego prawym synem jest NIL, w lewo dla minimum, jednak złożoność obliczeniowa taka sama jak find.

5 Omówienie przebiegu eksperymentów

5.1 Tabela wyników pomiarowych

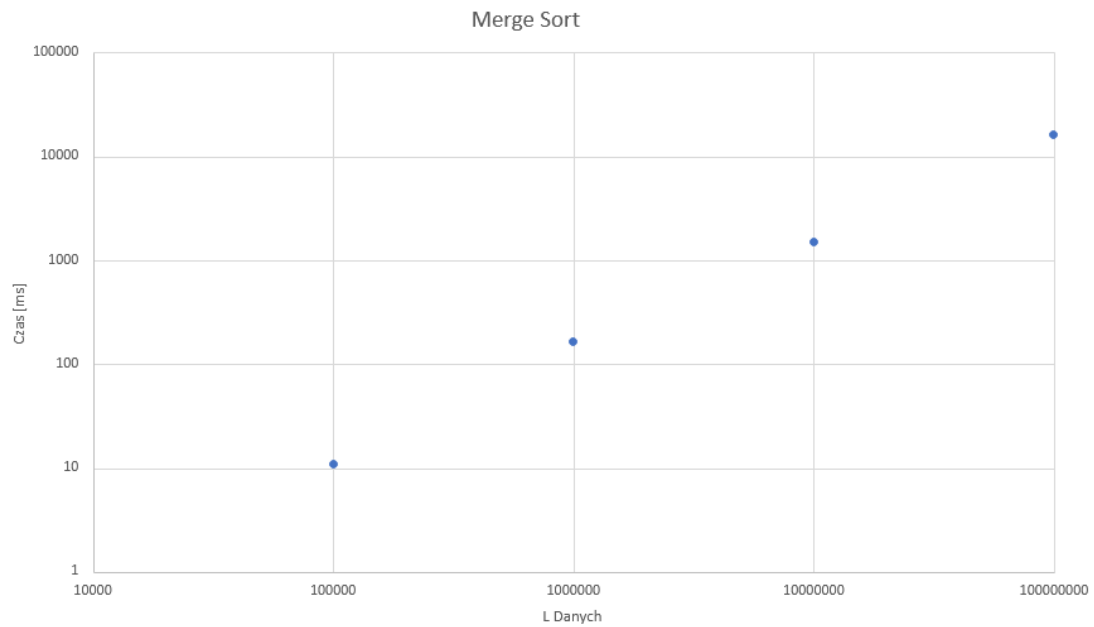
L Danych	BST	Merge	Quicksort(Duplikaty)	Quicksort	Insert	Mediana
10^5	2523	11	516	1	512	5.6
10^6	2721	164	8911	24	178650	6.3
10^7	6229	1486	3min+	5881	3min+	6.9
MAX -10mil	46426	16170		963		7.1

5.2 Drzewo czerwono-czarne



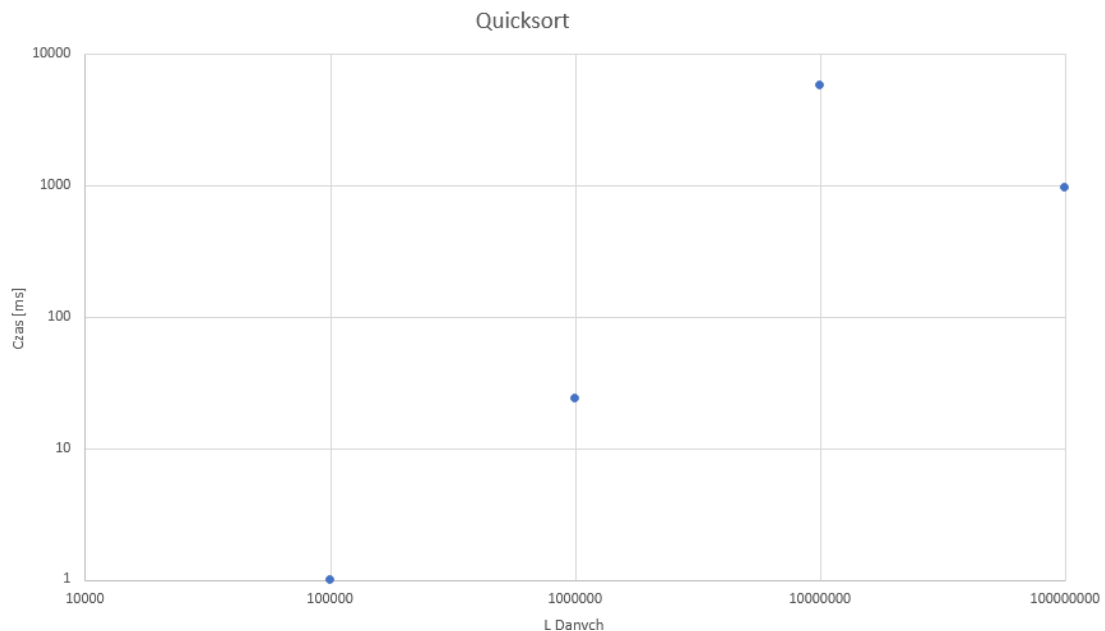
Wykres pomiarów drzewa czerwono-czarnego nie przypomina logarytmu, tak jak planowaliśmy, warto jednak wspomnieć, że pomiar dla maksymalnej liczby elementów miał miejsce na koniec, w związku z czym znaczącą rolę miała temperatura sprzętu - thermal throttling procesora. Ponadto złożoność obliczeniowa Red-Black BST zmienia się w zależności od liczby czerwonych node'ów, których liczby nie kontrolujemy.

5.3 Merge sort



W przypadku merge sortu wykres przypomina $n * \log n$

5.4 Quicksort



Quicksort przed zastosowaniem osobnej tablicy dla duplikatów nie był w stanie posortować liczby instancji 1 milion, wynika to z faktu, że ilość unikalnych ratingów jest ograniczona, tj. rating co 0.1 w przedziale od 1.0-10.0.

6 Bibliografia

- https://eduinf.waw.pl/inf/alg/001_search/0117.php
- https://eduinf.waw.pl/inf/alg/001_search/0121.php
- <https://www.algorytm.edu.pl/algorytmy-maturalne/sortowanie-przez-scalanie.html>
- <https://www.geeksforgeeks.org/dsa/merge-sort/>
- <https://www.geeksforgeeks.org/dsa/red-black-tree-definition-meaning-in-dsa/>
- <https://www.geeksforgeeks.org/cpp/std-remove-if-algorithm-in-cpp-stl/-usuwanieelementowzwektora>